

Linux, procesos, permisos y shell

Objetivos

Diferenciar usuario, proceso, shell, binario, biblioteca e intérprete; leer permisos; y predecir el efecto de quoting, expansiones, pipes y redirecciones.

Entidades distintas

- Usuario/grupo: identidades usadas por el kernel para comprobar permisos.
- Proceso: instancia en ejecución con PID, credenciales, entorno, directorio y descriptores.
- Binario: archivo ejecutable que contiene código o carga un intérprete.
- Biblioteca: código cargado por procesos; no es un proceso por sí misma.
- Shell: programa que lee, analiza, expande y ejecuta comandos.
- Parser/intérprete: funciones; Bash puede cumplir ambas, pero un programa puede recibir argumentos sin shell intermedia.

Esta diferencia decide si `;`, `$()` o `|` tienen significado. Si una aplicación ejecuta un binario con una lista de argumentos, esos caracteres pueden llegar como datos literales.

Permisos y credenciales

Los bits `rwx` se evalúan para propietario, grupo y otros. Directorios requieren `x` para atravesar y `w` para crear/eliminar entradas según permisos del directorio. SUID cambia el identificador efectivo al ejecutar un binario; sudo aplica una política; capabilities dividen privilegios. No son mecanismos equivalentes.

Pipeline de Bash

Según el manual de Bash, de forma simplificada:

1. tokeniza respetando quoting;
2. analiza comandos y operadores;
3. realiza expansiones;
4. aplica redirecciones;
5. ejecuta y recoge estado.

Expansiones relevantes: variables, sustitución de comandos, aritmética, separación de palabras y globbing. Las comillas simples inhiben expansiones; las dobles conservan algunas, como `$variable` y `$(comando)`.

Operadores

Forma	Función	Condición
<code>a; b</code>	ejecutar en secuencia	lo interpreta una shell
<code>a && b</code>	ejecutar b si a devuelve 0	shell
<code>a b</code>	ejecutar b si a falla	shell
<code>a b</code>	stdout de a a stdin de b	shell crea pipe
<code>>, >>, <</code>	redirigir descriptores	shell
<code>\$(...)</code>	sustituir por salida	expansión habilitada

Argumentos y opciones

La shell produce una lista de argumentos. Después, el programa decide si un argumento que empieza por - es opción. -- suele indicar fin de opciones, pero solo si la herramienta lo implementa. Command injection altera la gramática de shell; argument injection conserva el proceso pero altera su lista de argumentos.

Evidencia

Para comprender un fallo en laboratorio, registra usuario real/efectivo, proceso padre, comando o argv, entorno, directorio y permisos de los archivos implicados. "Tengo una shell" no implica TTY completa, permisos elevados ni canal estable.

Experimento: distinguir shell de argv

Una aplicación ejecuta una comprobación de host. Con host=127.0.0.1; id la respuesta muestra literalmente ping: 127.0.0.1; id: Name or service not known.

Observación: ; llegó a ping, pero no separó comandos. Qué sé realmente: el carácter no fue interpretado como operador por una shell antes de llegar al programa. Hipótesis: H1, ejecución directa `execve("ping", ["ping", "-c", "1", entrada])`; H2, shell con el valor correctamente citado. Experimento: usar una entrada que provoque un error propio de opciones, como -h, y observar argv en un wrapper de laboratorio; después comparar con una variante que contenga expansión `$(printf X)`. Predicción: H1 conserva cada cadena como un único argumento; H2 puede expandir solo si el quoting lo permite. Resultado: el log muestra `argv[3]="127.0.0.1; id"`. Conclusión: no hay command injection demostrada; todavía podría existir option injection según cómo se construya argv.

Construcción por capas

Para POST {"host": "127.0.0.1; printf MARK"}, separa siempre:

1. comillas y escapes que hacen JSON válido;
2. el valor resultante entregado por el deserializador;
3. quoting añadido por la aplicación, si existe;
4. gramática de shell o lista argv final;
5. canal donde debería aparecer MARK.

Un 400 JSON refuta la llegada al shell; un error de ping confirma llegada a ping, no ejecución de un segundo comando.

Referencias

- [GNU Bash Reference Manual](#)
- [Conceptos internos de escalada y TTY](#)

Anterior: [Encoding](#). Siguiendo: [Bases de datos y flujo](#).